

Distributed Systems - 20_10

Let's now see the proof of the algorithm.

At first, we take a look at a statement: If a correct process p sees a message m , it will eventually deliver it.

What I mean by "sees a message"? That a correct process has received the message.

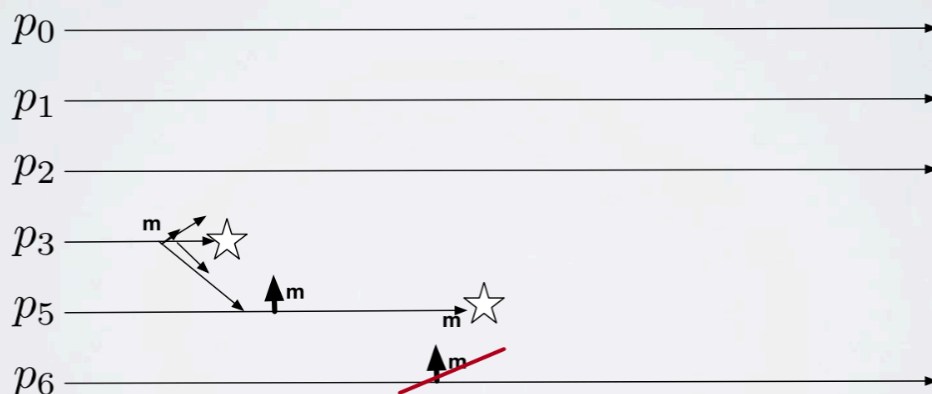
proof: By assumption p is correct, thus its message will reach every other correct process (see BEB). By the strong completeness of FD P eventually process p detects as crashed all the crashed processes, thus it will not wait forever for an ack of a crashed process. Therefore, $\text{candeliver}(m)$ on p will be true.

This statement proves the validity property.

Now we need to prove the others. No duplication and no creation are trivial.

So we have to show the uniform agreement property.

Agreement proof: By contradiction. Suppose p_5 (faulty) delivers m and p_6 correct does not. The only possibility is that p_6 does not see the message (see previous slide). This implies that p_6 is detected faulty by p_5 , p_5 delivers without receiving the ack from p_6 . This contradicts the strong accuracy of the failure detector P .



We don't take a look at the case in which a correct process receives a message and not deliver it, because of the statement we've shown before.

Basically p_5 is delivering the message without receiving the ack from p_6 , and the only possible way this happens is because p_6 is detected as dead by P . Though p_6 is correct, so we're breaking the strong accuracy of P .

What is the message complexity of URB? In the best case it's 2 (we send the broadcast, and the other process relays the message). In the worst case, we need $N - 1$ messages (due to a chain of failures).

Can we build this algorithm without P ? We want to build it in Fail-Silent.

We'll use Quorum. It's a subset of Π (Π with a length of n) of size at least $\frac{n}{2} + 1$.
(if we have a set of 10 processes, then a quorum is a subset of length at least 6).

We have the following property:

- Any two quorums, intersect in at least one process.

The proof is by contradiction, we suppose two quorums do not intersect, then they all have distinct processes. But then this means that $|\Pi| > n$.

Why are quorums important? With them we can be sure that at least one correct process has seen our message.

The assumption that we use is the "minority of failures", so failures are a minority in a system. This means that the number of failures is $f < \frac{n}{2}$. So, the number of correct process will be always greater than the number of faulty processes. The number will always be $\geq \frac{n}{2} + 1$, so the set of correct processes is a quorum. Even if we intersect with another subset of Π (which could also contain faulty processes), there will be at least one correct process.

Now, I want to build URB in Fail-Silent assuming that the correct processes are $\geq \frac{n}{2} + 1$.

What could be the idea? I know that if one correct process has seen the message, then I can deliver it safely (that correct process will send back the message to everyone).

We send messages to everyone in the system, and start receiving ACK. When I receive a number of ACK greater and equal than $\frac{n}{2} + 1$, i'm fine.

We modify the original algorithm by removing the Crash handler and modifying the function `candeliver(m)`. We have that when $|ack(m)| > \frac{n}{2}$ we can deliver.

Now, suppose we have a set of n processes, and I have $f = \frac{n}{2} + 1$. (like 6 processes and 4 failures).

We modify the algorithm by saying that `candeliver(m)` is true when $|ack[m]| > f$.

Why is this algorithm not correct? We could have 6 processes and $p_1 \dots p_4$ are faulty. Then we have p_5 and p_6 . If p_5 (correct) sends a message, then p_6 has to deliver it (validity). We say that everyone sees the message, but then from p_1 to p_4 will die. We'll have that p_5 will only see two ACKs (p_5 and p_6), so he will never deliver.

Another counter example is p_5 receiving the ACK from everyone, and so he will deliver, but p_6 does not receive them, then p_6 will not deliver.

This was a situation where our assumption is broken, because now we have a number of faulty processes greater than $\frac{n}{2}$.

The URB is important because it shows that we can build the URB with the P , and also without it when we have the majority of correct processes.

The RB, we can build it with P and even without it, but we don't need the majority of processes.

The assumption in URB in fail-silent is necessary, without it we can't build it.

So URB is a problem we can solve in Fail Stop, but not in Fail Silent (without an assumption). We've seen how Fail Stop and Fail Silent are different.

Let's do an exercise.

Exercise 3.5: Consider the “All-Ack Uniform Reliable Broadcast” algorithm (Algorithm 3.4). What happens if the strong accuracy property of the perfect failure detector is violated? What if its strong completeness property is violated?

Algorithm 3.4: All-Ack Uniform Reliable Broadcast

Implements:

UniformReliableBroadcast, instance *urb*.

Uses:

BestEffortBroadcast, instance *beb*.

PerfectFailureDetector, instance $\mathcal{P}$.

upon event $\langle \text{urb}, \text{Init} \rangle$ **do**

delivered := $\emptyset$;

pending := $\emptyset$;

correct := Π ;

forall *m* **do** *ack*[*m*] := $\emptyset$;

upon event $\langle \text{urb}, \text{Broadcast} \mid m \rangle$ **do**

pending := *pending* $\cup \{(self, m)\}$;

trigger $\langle \text{beb}, \text{Broadcast} \mid [DATA, self, m] \rangle$;

upon event $\langle \text{beb}, \text{Deliver} \mid p, [DATA, s, m] \rangle$ **do**

ack[*m*] := *ack*[*m*] $\cup \{p\}$;

if $\langle s, m \rangle \notin \text{pending}$ **then**

pending := *pending* $\cup \{(s, m)\}$;

trigger $\langle \text{beb}, \text{Broadcast} \mid [DATA, s, m] \rangle$;

upon event $\langle \mathcal{P}, \text{Crash} \mid p \rangle$ **do**

correct := *correct* $\setminus \{p\}$;

function *candeliver*(*m*) **returns** Boolean **is**

return (*correct* $\subseteq \text{ack}[m]$);

upon exists $\langle s, m \rangle \in \text{pending}$ such that *candeliver*(*m*) $\wedge m \notin \text{delivered}$ **do**

delivered := *delivered* $\cup \{m\}$;

trigger $\langle \text{urb}, \text{Deliver} \mid s, m \rangle$;

For the first question, we'll have that, if strong accuracy is violated, then a process is detected as faulty even though is not. We reason by thinking: "is it possible we violate no duplication?", no, there is the set checking it, also this applies to no creation. The only property we could violate is Validity and Agreement. For agreement, we should try to remember the proof, checking a situation that violates the property.

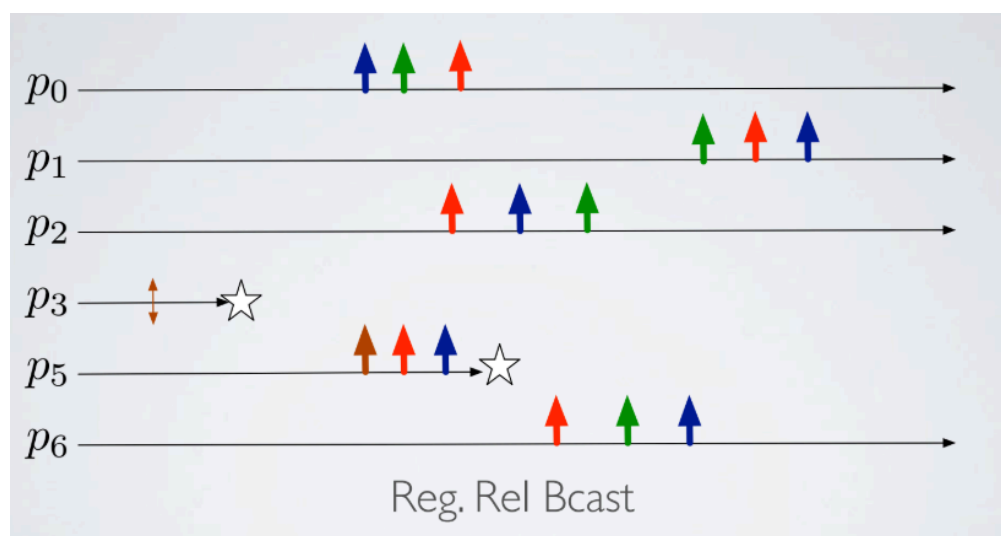
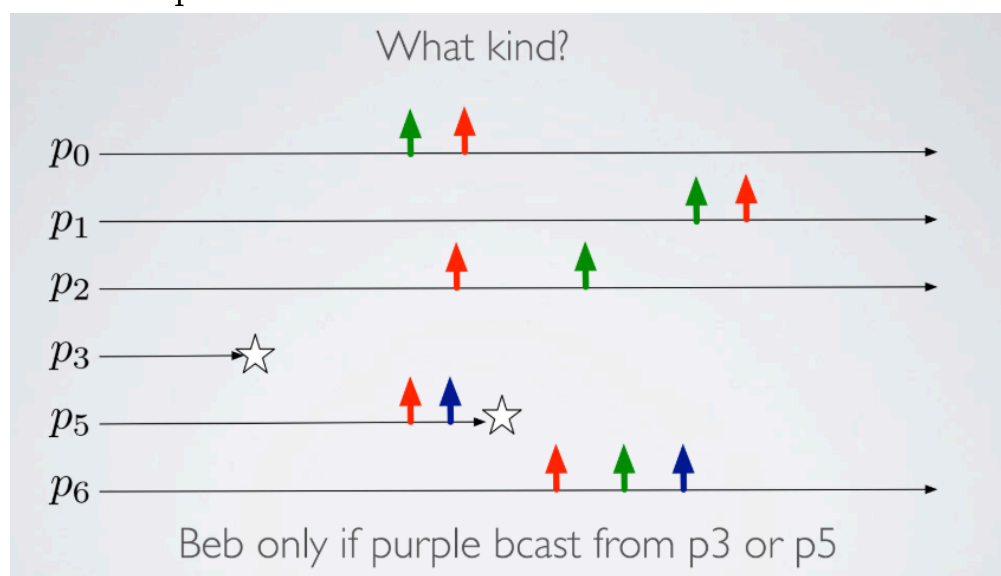
How do we have p_5 deliver the message before receiving an ACK from p_6 ? Because P is not strongly accurate, we may have the relaying from p_5 . Then I receive the signal p_0 dead, and then also with p_1, p_2 and p_6 . Once I receive all this "assumptions" made by P (not working properly), then it happens that *candeliver*(*m*) becomes true. To finish the counter example, I have to destroy the messages relayed from p_5 , they might reach a correct process and then anything could happen.

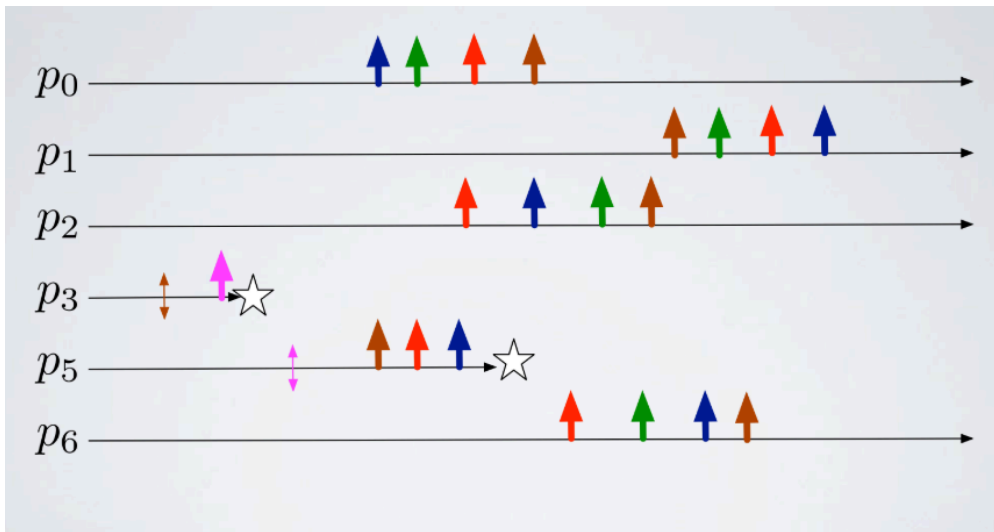
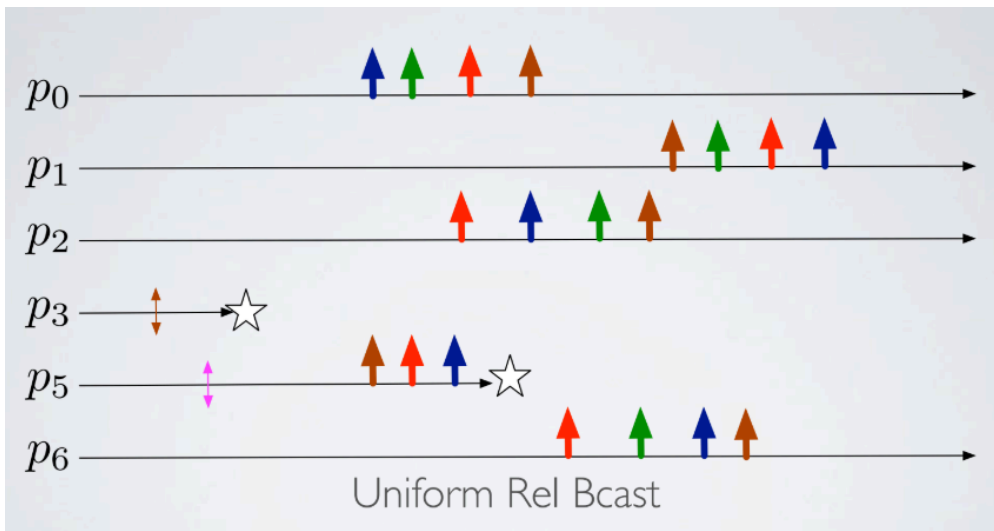
Why I cannot violate validity? To violate validity, I need a process that is correct, broadcasts a message and never delivers it. Remember that P violates S.A., so there is a set of faulty process and also some correct processes. The only way to not deliver a message is to never reach a correct $\subseteq ack[m]$. So there must be a process that is faulty in the system, and we wait for his ack forever (we've not detected a process as faulty). Can this happen? No. *Check this thing better*

What happens now if P is not strong complete. We'll violate validity. What is the counter example? We have two processes, p_1 and p_2 . p_1 broadcasts, the message arrives at p_2 but then it crashes. p_1 will never detect p_2 as dead (because P is faulty), and so we'll wait for the ACK forever, never delivering the message.

Order Communication

Small recap:





This last one is not a correct broadcast because the pink message is being delivered before being even broadcast.

Until now, we didn't care about the order, but it actually matters.

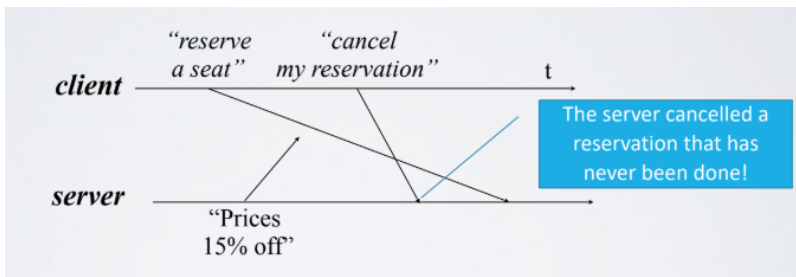
We'll take a look at different types of ordering:

- FIFO ordering, the same we've seen on the link. The delivery of the messages from the same sender respect the ordering of the sending of messages
- Causal ordering, the ordering is dictated by an "happened before" relationship
- Total ordering, also known as atomic communication

Usually, total order does not imply FIFO and causal ordering.

The ordering is orthogonal, so a property of the ordering does not imply other properties.

Imagine an example:



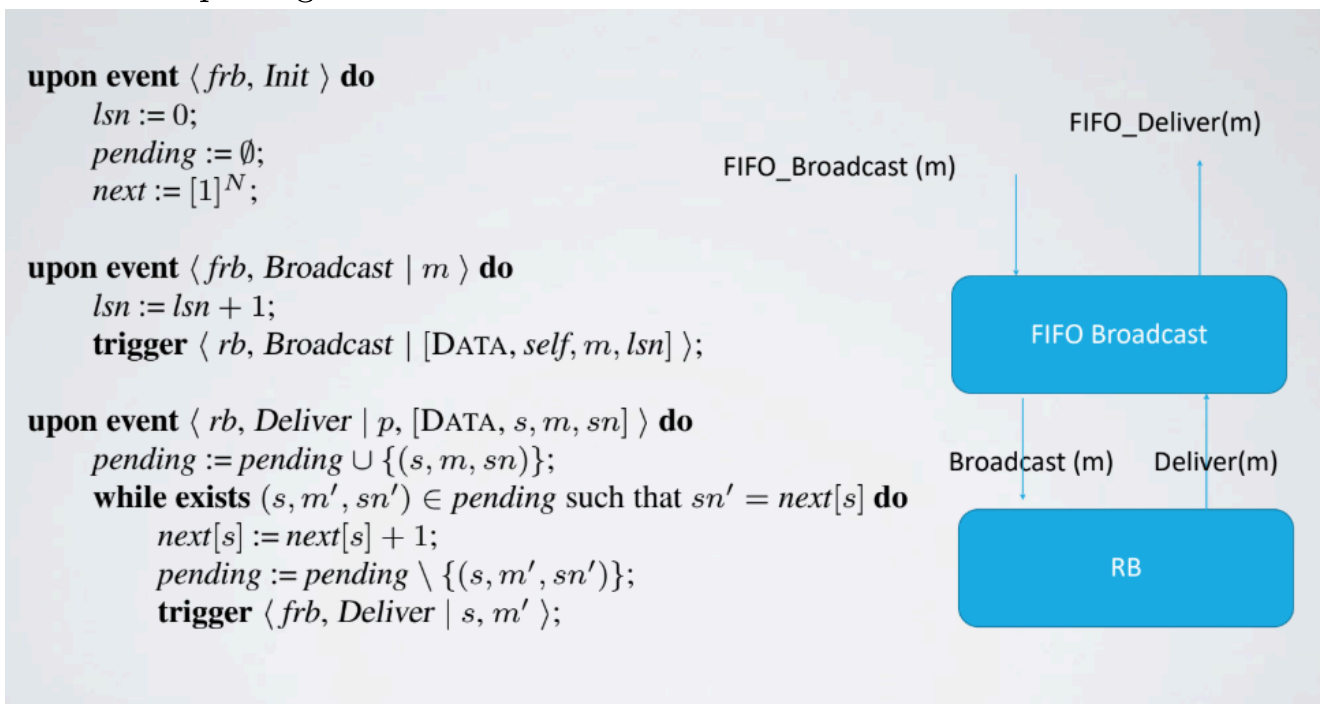
To fix this example, we can simply use the FIFO ordering.

We have a FIFO (reliable) broadcast module.

It shares the same properties of the RB, with an additional one:

- (FIFO delivery): If some process broadcast message m_1 before it broadcast message m_2 , then no correct process delivers m_2 , unless it has already delivered m_1 .

How do we implement this algorithm? With the exact same algorithm seen in the link, we use a sequence number for messages, and deliver only if the number is the one we're expecting.



lsn is the sequence number, then we have a *pending* set, and also $next := [1]^N$, so an array of length N (N = number of processes), that starts with 1 for each process. Each location is the sequence number from a certain sender.

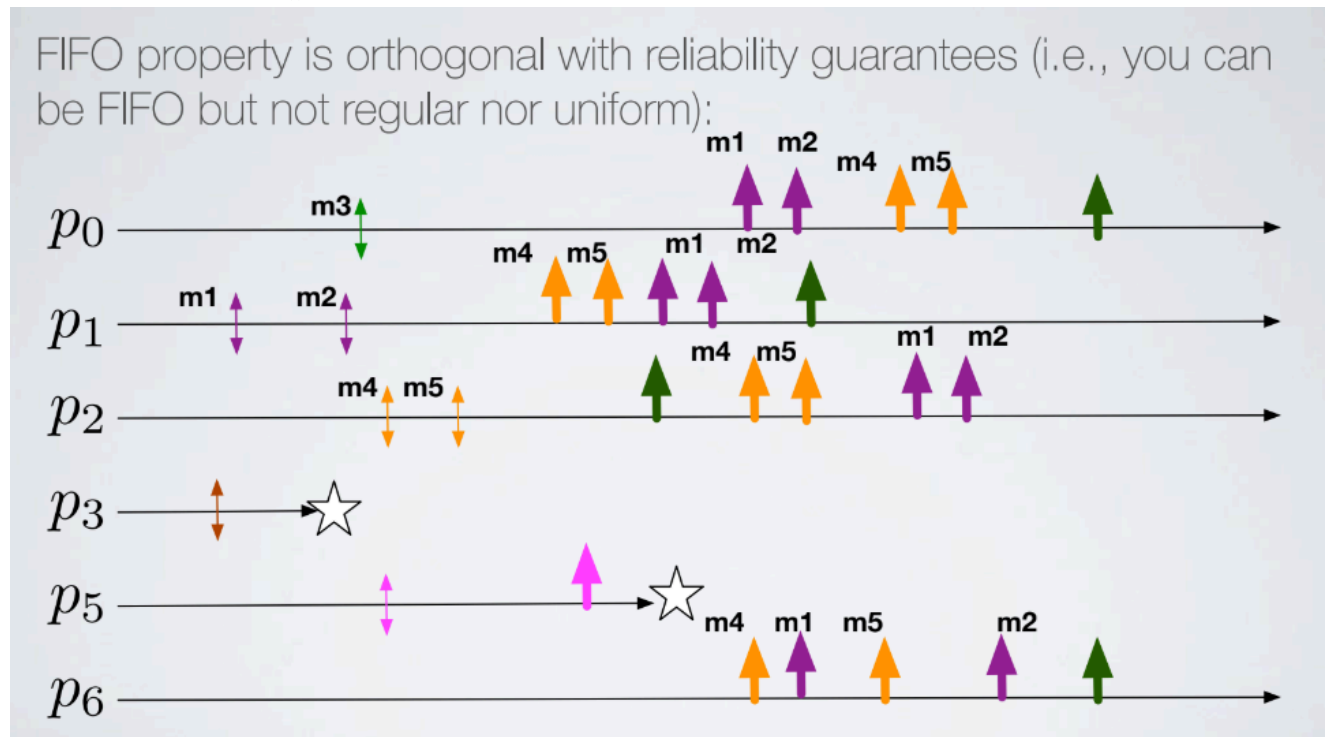
When we send a message, we increase the sequence number, and then broadcast the message to the RB.

When a process delivers a message, he appends the message to the pending set, then checks with a while if there is a tuple of $(s, m', sn') \in pending$ with the $sn' = next[s]$. So we are checking, for a certain process s , if there is a message with the correct sequence number that we have to deliver. If that's true, we

increase the sequence number for that process, remove the message from pending set and trigger the delivery in the FIFO reliable link.

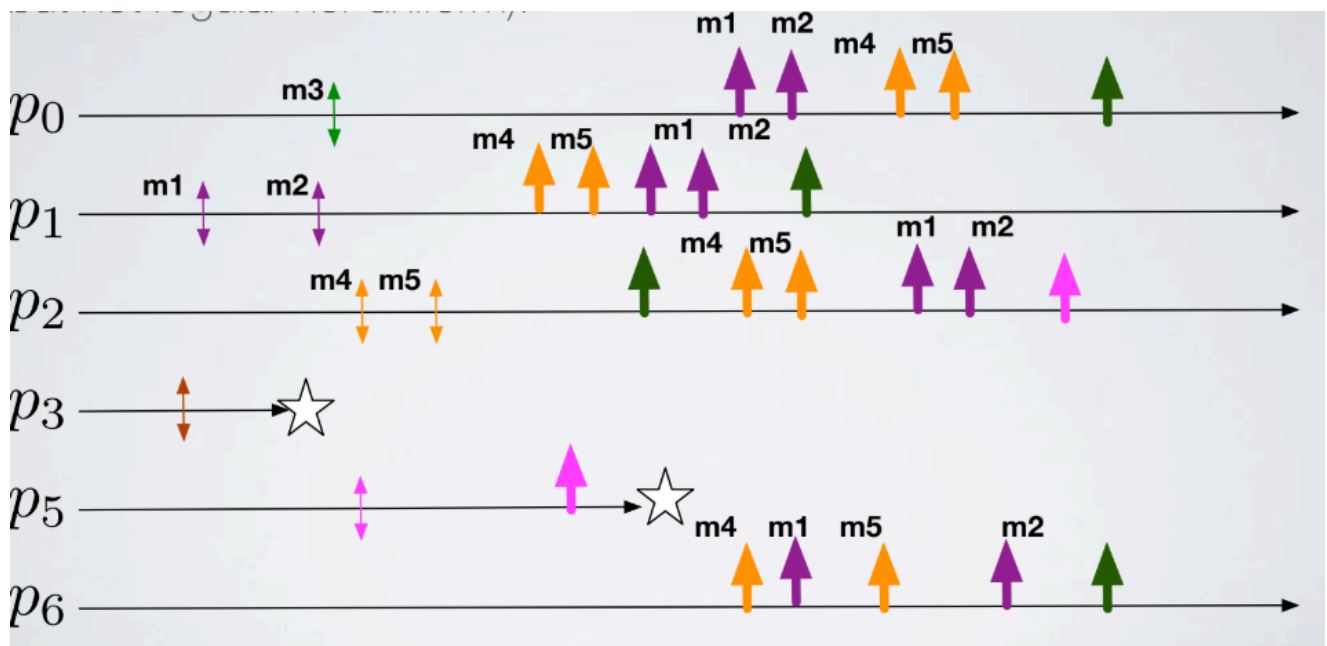
With this kind of ordering, there is no ordering from messages coming from different senders.

Let's see an example:



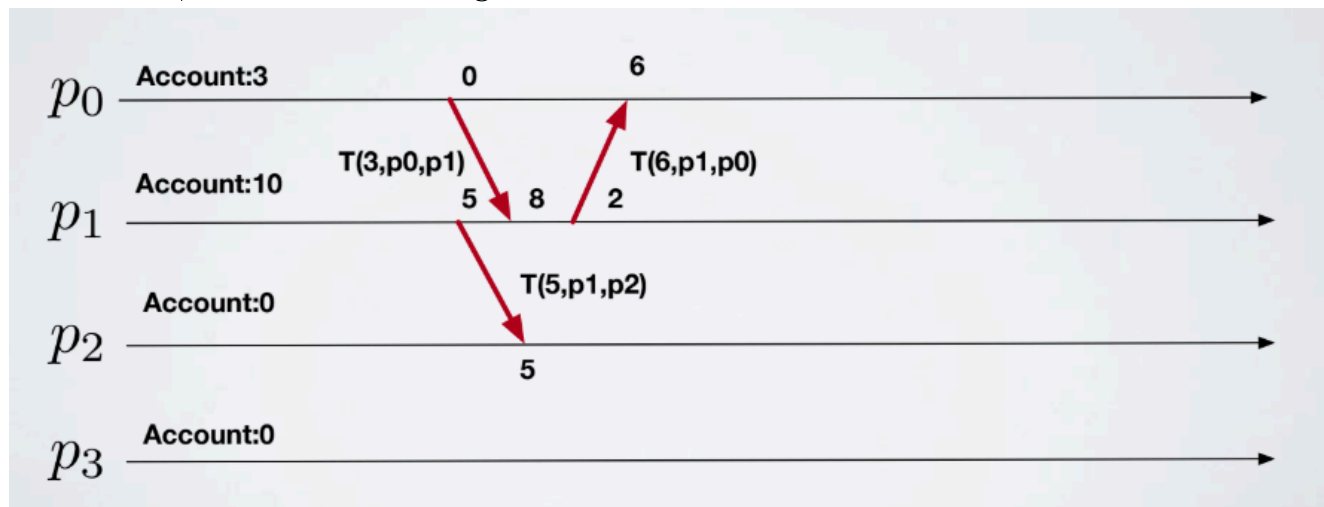
Here it's RB.

While here:



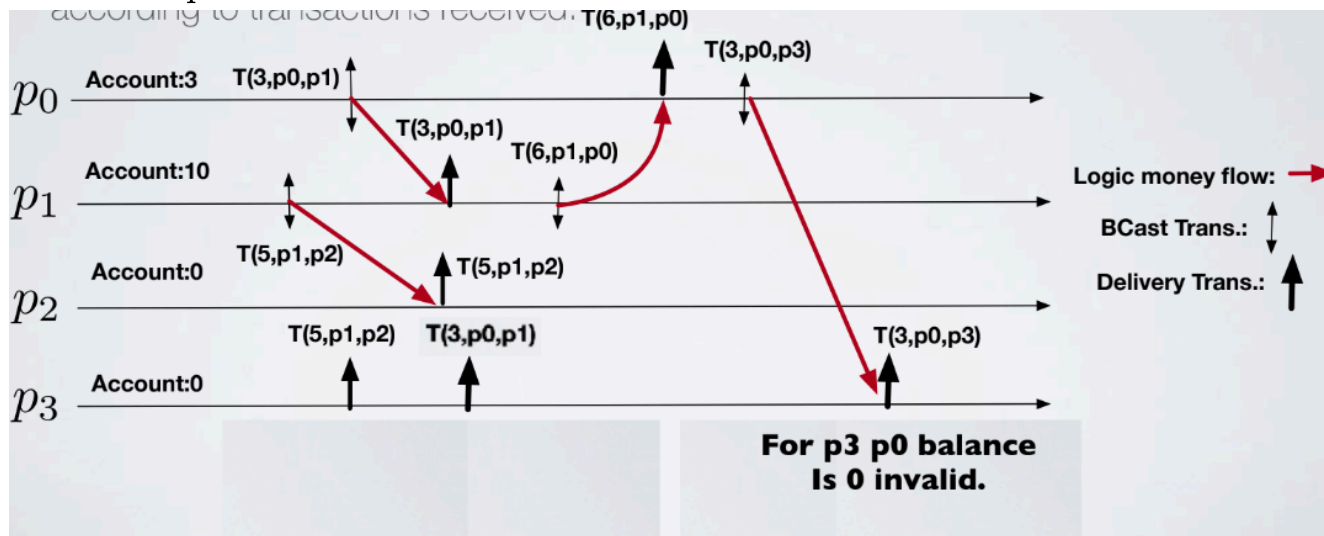
It's BEB.

Sometimes, FIFO is not enough.



For this example, we want to start a transaction with a broadcast message. Everyone that sees the message, update the account of the other processes. There could be the possibility, however, that one process wants to transfer more money than he has, and then the transaction is not valid even if the process signed the transaction.

In the following example, if we used an happened before relationship, we would not have a problem:



Notice that some transactions are concurrent, so p_3 could receive transactions in different order, but the system would work anyway.

So we use the casual order broadcast. If there are two message m_1 and m_2 , and m_1 happens before m_2 , then no process will deliver m_2 unless he delivered m_1 .

For the primitive, it's called Casual Order Reliable Broadcast. The events are the same, as well as the first four properties of RB. We also have:

- (Casual Delivery): For any message m_1 that potentially caused a message m_2 ($m_1 \rightarrow m_2$), no process delivers m_2 unless it has already delivered m_1 .

How do we implement this? When I create a message, I send the history of the messages that caused that message.

Exercises

Consider a set of n processes connected with a ring topology. These processes have unique IDs taken from an unknown set (that is not the set $\{1, \dots, n\}$). Each process is correct and cannot crash. Processes are connected by perfect point2point links.

Q1: Write an algorithm that elects a leader without knowing n .

Q2: What is the time and message complexity of the algorithm?

What is the message complexity in the best case?

Q3: Consider now that you are in a synchronous system with point2point link with a guarantee on delivery time (Δ max delivery time) and knowledge of n . Can you devise an algorithm with a linear message complexity?

We could have a discovery phase in which each process sends a message to the right, mapping the IDs, until the message reaches the original sender. Then everyone will have a map of the processes, and could proceed with the leader election,

We'll need $n \times n$ messages to map, and then the same complexity of le .

Another version is going twice around the ring topology.

Another solution could be that, at the beginning, we have a leader variable that I set to false, that will be set to true when we have elected a leader. I also have a variable `leader_id`, initially set to my Id. Then a process will send to its right neighbor his candidate, and the neighbor will take the minimum value. This cannot work this way because we're in asynchronous system. We would have stopped when we received my id, but one link could be super fast and other super slow.

So, to fix this, we could kill a message in which the `leader_id` is greater than ours.

Let's now write the algorithm.

```
upon Init
  let right_neighbour
```

```

    leader = false
    leader_id = $p_i$
    send_message(right_neighbour, CANDIDATE, p_i)

upon message from p_x (candidate, p_i)
    if leader_id > p_i then
        leader_id = p_i
        send_message(right_neighbour, CANDIDATE, leader_id)
    else if leader_id = p_i then # message had finished the loop
back
        leader = true
        send_message(right_neighbour, LEADER, leader_id)
        # if the id we receive is greater than our candidate, we
ignore

upon message from p_x (LEADER, leader_id) do
    if leader_id != p_i then
        leader = true
        send_message(right_neighbour, LEADER, leader_id)

```

Let's now do question 2, by taking a look at the message complexity.

It's $\sum_{i=1}^n i = \frac{n(n-1)}{2}$ in the worst case, plus n for the leader. In the best case, $2n$.

🔗 Exercise

Do the last question